Cloud Matrix (AWS Calcus): Project Documentation

> *Naming note: as requested, the company's first-word prefix has been removed throughout this document. The company name reads "Global Software Pvt Ltd" and the demo e-mail domain reads `global.com`. The code's internal package and folder name is `aws-calcus` ("AWS Calcus"); it is kept where it is a real file or package name.*

1. Overview and purpose

Cloud Matrix is an internal enterprise web application that lets employees estimate and compare the cost of running the same infrastructure on **AWS**, **Azure** and **Google Cloud (GCP)**.

A user selects compute instances and storage volumes, then a commitment duration. The server prices the selection for each provider, marks the cheapest one, stores the result in a per-user history and returns it. The browser shows three provider cards and a stacked bar chart, and can export a PDF report or an Excel sheet. Administrators manage users, read the audit trail and see system statistics.

Key capabilities:

- Email and password login, JWT tokens valid for 24 hours, two roles (`admin`, `user`).
- Four-step calculator wizard: Compute, Storage, Duration, Results.
- Multi-cloud comparison with cheapest-provider highlighting.
- PDF and Excel export (generated in the browser).
- Paginated calculation history with delete and re-export.
- Admin panel: statistics, user creation and deletion, audit log viewer, ingestion trigger button.
- Audit logging of logins, calculations and user management, with client IP.

2. Tech stack

Area	Technology	Notes
Language	TypeScript 5.9	Project references via `tsconfig.base.json`
Runtime	Node.js 24	Declared in `.replit`
Package manager	pnpm workspaces	`preinstall` script rejects npm and yarn
Frontend	React 19.1, Vite 7, Tailwind CSS 4	Dark glass-style theme, fonts Inter and Syne
UI kit	shadcn/ui on Radix	About 55 generated components
Routing	wouter	Base path from `BASE_URL`
Server state	TanStack Query 5	`retry: 1`, no refetch on focus
Forms	react-hook-form with Zod resolver	Login and user management
Charts / animation	Recharts, framer-motion	Results chart, wizard transitions
Export	jsPDF, jspdf-autotable, xlsx (SheetJS)	Client side
Backend	Express 5, cors, jsonwebtoken, bcryptjs	HS256 token, bcrypt cost 12
Database	PostgreSQL with `pg` pool	Drizzle ORM 0.45, drizzle-kit `push`
API contract	OpenAPI file with Orval codegen	React Query hooks and Zod schemas
Build	esbuild (API bundle), Vite (web)	API output `dist/index.cjs`
Hosting target	Replit autoscale	Original brief asked for an on-premises Linux VM

3. Repository structure

```
Cloud-Matrix/
  package.json, pnpm-workspace.yaml, tsconfig*.json, .replit, replit.md
  attached_assets/       two identical copies of the original product brief
  artifacts/
    api-server/          Express API
```

```
    build.ts           esbuild production bundle
    src/index.ts       reads PORT and starts the server
    src/app.ts         cors, JSON/urlencoded parsers, mounts /api
    src/lib/auth.ts    JWT, bcrypt, requireAuth, requireAdmin
    src/lib/audit.ts   logAudit()
    src/routes/        health, auth, providers, pricing, calculations, admin
  aws-calcus/          React web app
    src/App.tsx        providers, router, ProtectedRoute
    src/components/    layout.tsx (shell), ui/ (shadcn)
    src/hooks/         use-auth, use-toast, use-mobile
    src/lib/           export-utils.ts, utils.ts
    src/pages/         login, dashboard, calculator, history, not-found, admin/*
  mockup-sandbox/      design-preview Vite app (not part of the product)
lib/
  api-spec/            openapi.yaml, orval.config.ts
  api-client-react/    generated hooks, custom-fetch.ts
  api-zod/             generated Zod schemas and types
  db/                  Drizzle schema, pool, drizzle.config.ts
scripts/
  src/seed.ts          seeds users, providers, regions and prices
  post-merge.sh        pnpm install and schema push
```

Workspace packages: `@workspace/api-server`, `@workspace/aws-calcus`, `@workspace/mockup-sandbox`, `@workspace/db`, `@workspace/api-spec`, `@workspace/api-client-react`, `@workspace/api-zod`, `@workspace/scripts`. The pnpm `catalog:` block pins shared versions (React, Vite, Tailwind, Drizzle, Zod) for all packages.

4. Architecture

```
Browser (employee / admin)
      |  HTTPS: static files at "/" and JSON calls at "/api"
      v
+---------------------+  /api/v1/*  +-----------------------+
| React + Vite web app| ----------> | Express 5 API server  |
+---------------------+  Bearer JWT +-----------+-----------+
                                                | node-postgres pool
                                                v
                                      +---------------------+
                                      | PostgreSQL (Drizzle)|
                                      +---------------------+
```

Layers:

Layer	Responsibility	Location
Presentation	Pages, forms, charts, exports, route guards	`artifacts/aws-calcus`
API client	Typed hooks generated from OpenAPI	`lib/api-client-react`
Contract	Endpoints and payload shapes	`lib/api-spec/openapi.yaml`
Application	Routing, JWT auth, role checks, audit	`artifacts/api-server/src`
Core logic	Price lookup, per-provider totals, cheapest detection	`routes/calculations.ts`
Data	Tables, relations, pool	`lib/db`

Contract-first flow: the OpenAPI file is run through Orval to produce React Query hooks and TypeScript types (`lib/api-client-react/src/generated`) and Zod schemas (`lib/api-zod/src/generated`). All generated calls go through `customFetch`, which sets JSON headers, merges caller headers, parses JSON, text or blob responses by content type, and throws a typed `ApiError`. The web app never builds URLs by hand.

Sign-in flow

```
Browser                    API                              PostgreSQL
  POST /api/v1/auth/login    |                                  |
  ------------------------->| SELECT user by lower(email)       |
                            | bcrypt.compare(password, hash)    |
                            | UPDATE users.last_login           |
                            | INSERT audit_logs (login)         |
  <-------------------------| {user_id,email,role,token,expiry} |
  token saved in localStorage key "calcus_auth"; redirect "/"
```

Calculation flow

```
Wizard                     API                                PostgreSQL
  POST /api/v1/calculations   | requireAuth (verify JWT)
```

```
 {compute_selections,        | load providers (id -> name)
  storage_selections,        | per compute selection: SELECT price row by id,
  duration_months}           |    monthly = price_per_month * quantity
 -------------------------->| per storage selection: SELECT price row by id,
                            |    monthly = price_per_gb_month * size_gb
                            | sum per provider, flag cheapest
                            | INSERT calculations (JSON snapshots + totals)
                            | INSERT audit_logs (calculate_pricing)
 <--------------------------| 201 CalculationResult
```

5. Detailed component walkthrough

5.1 API server (`artifacts/api-server`)

- `index.ts` requires the `PORT` environment variable (it throws if missing or invalid) and calls `app.listen`.
- `app.ts` installs `cors()` with default (open) settings, `express.json()`, `express.urlencoded`, and mounts the router at `/api`.
- `lib/auth.ts` exports `generateToken`, `verifyToken`, `hashPassword`, `comparePassword`, `requireAuth` and `requireAdmin`. The token payload is `{userId, email, role}` signed with `JWT_SECRET` (falls back to a hard-coded default), expiry `24h`. `requireAuth` expects `Authorization: Bearer <token>` and answers `401` with `{error, message}` otherwise. `requireAdmin` answers `403` unless the role is `admin`.
- `lib/audit.ts` exports `logAudit({userId, action, inputData, status, errorMessage, req})`. It stores the first value of `x-forwarded-for` (or the socket address) as the IP. It catches its own errors so auditing never breaks a request.

5.2 REST endpoints

Base path `/api`. Every route except login and health requires a token.

Method	Path	Access	Purpose
GET	`/healthz`	public	Returns `{status: "ok"}` (used as deploy health check)
POST	`/v1/auth/login`	public	Email and password to `{user_id,email,role,token,expires_at}`
POST	`/v1/auth/register`	admin	Creates a user and returns a token for that user
GET	`/v1/auth/me`	user	Current user profile
GET	`/v1/providers`	user	AWS, Azure, GCP
GET	`/v1/providers/{providerId}/regions`	user	Regions of one provider, ordered by name
GET	`/v1/pricing/compute`	user	Filters: `provider_id`, `region_id`, `os_type`; ordered by monthly price
GET	`/v1/pricing/storage`	user	Filters: `provider_id`, `region_id`, `storage_type`
POST	`/v1/calculations`	user	Runs and stores a calculation
GET	`/v1/calculations`	user	Paginated list (`page`, `limit` default 20, `from_date`, `to_date`); admins see all users' rows
GET	`/v1/calculations/{id}`	owner or admin	One calculation
DELETE	`/v1/calculations/{id}`	owner or admin	Deletes it and writes an audit entry
GET	`/v1/admin/users`	admin	All users, newest first
POST	`/v1/admin/users`	admin	Create a user (email, password, optional role)
PUT	`/v1/admin/users/{id}`	admin	Update email, role or password
DELETE	`/v1/admin/users/{id}`	admin	Delete a user; deleting your own account is refused

Method	Path	Access	Purpose
GET	`/v1/admin/audit-logs`	admin	Paginated (`page`, `limit` default 50), filters `user_id`, `action`; joins the user's email
GET	`/v1/admin/stats`	admin	Counts, last ingestion time, active users in last 30 days
POST	`/v1/admin/ingest`	admin	Writes an audit entry and returns a success message (stub)

Error shape for all failures: `{ "error": "<Reason>", "message": "<detail>" }` with status 400, 401, 403 or 404.

Response conventions: JSON field names are snake_case. Numeric database columns (strings in PostgreSQL) are converted with `parseFloat` before returning.

5.3 Database (`lib/db`)

Connection: `DATABASE_URL` is required; the module throws at start-up without it. `db` is a Drizzle instance over a `pg` Pool. The schema is applied with `drizzle-kit push` (no migration files).

Enums: `role` (admin, user), `audit_status` (success, failure), `storage_type` (block, object).

Table	Main columns	Notes
`users`	id, email (unique), password_hash, role, created_at, last_login	E-mail stored lower-case
`providers`	id, name (unique), created_at	AWS, Azure, GCP
`regions`	id, provider_id, region_code, region_name, created_at	FK to providers
`compute_pricing`	id, region_id, provider_id, instance_type, os_type, price_per_hour (10,6), price_per_month, price_per_year, vcpu, memory_gb, updated_at	FKs to regions and providers
`storage_pricing`	id, region_id, provider_id, storage_type, storage_name, price_per_gb, price_per_gb_month (10,6), unit_type, updated_at	The `price_per_gb_month` value is used for costing
`calculations`	id, user_id, input_json, result_json, cheapest_provider, aws/azure/gcp_total_monthly (10,2), duration_months, created_at	JSON snapshots plus denormalised totals
`audit_logs`	id, user_id (nullable), action, input_data (json), status, error_message, ip_address (45), timestamp	`input_data` is never populated by the code

Relations: users 1..n calculations; users 1..n audit_logs; providers 1..n regions; regions 1..n compute_pricing and storage_pricing (provider_id is also stored directly on pricing rows). `calculations` has no foreign key to pricing rows because it stores snapshots.

Audit action names used: `login`, `calculate_pricing`, `delete_calculation`, `create_user`, `update_user`, `delete_user`, `trigger_ingestion`.

5.4 Seed data (`scripts/src/seed.ts`)

Run with `pnpm --filter @workspace/scripts run seed`.

- Users: `admin@global.com` / `admin123` (admin) and `user@global.com` / `user123` (user), with bcrypt cost 12.
- Providers: AWS, Azure, GCP. Five regions each (AWS: us-east-1, us-west-2, eu-west-1, ap-south-1, ap-southeast-1; Azure: eastus, westus2, westeurope, southeastasia, centralindia; GCP: us-central1, us-west1, europe-west1, asia-southeast1, asia-south1).
- Compute catalogue: AWS 13 types (t2, t3, m5, c5, r5; Linux and Windows), Azure 12 (Standard_B, D, F, E series), GCP 12 (e2 family and others). The same list is inserted into every region of a provider at the same price. Total 185 rows.
- Storage catalogue: 6 per provider (3 block, 3 object), 5 regions each, 90 rows. Example monthly prices per GB: AWS gp3 0.080, S3 Standard 0.023; Azure Premium SSD P10 0.126, Blob Hot 0.018; GCP SSD Persistent Disk 0.170, Cloud Storage Standard 0.020.

- The pricing tables have no unique constraint, so `onConflictDoNothing` has no effect there: running the seed twice duplicates all price rows.

5.5 Web app (`artifacts/aws-calcus`)

Routing (`App.tsx`): `/login` is public. `/`, `/calculator` and `/history` need a signed-in user. `/admin`, `/admin/users` and `/admin/audit-logs` need an admin. Any other path shows the not-found page. `ProtectedRoute` redirects unauthenticated users to `/login` and non-admins to `/`.

Authentication state (`hooks/use-auth.tsx`): `AuthProvider` keeps the login response in React state and in `localStorage` under `calcus_auth`. At start-up an expired `expires_at` clears it. It exposes `token`, `user`, `login`, `logout`, `isAuthenticated`, `isAdmin`. `useAuthHeaders()` returns `{headers: {Authorization: "Bearer <token>"}}`, which every page passes to the generated hooks as `request`.

Layout (`components/layout.tsx`): a fixed desktop sidebar (brand, nav links, user menu with log out) and a mobile header with a slide-out sheet. Admin links appear only for admins.

Pages:

Page	File	Behaviour
Login	`pages/login.tsx`	react-hook-form login; calls `useLogin`; toast on result; redirects to `/`
Dashboard	`pages/dashboard.tsx`	Welcome message; for admins four stat cards; the five latest calculations
Calculator	`pages/calculator.tsx`	The wizard described below
History	`pages/history.tsx`	10 rows per page, per-row PDF and Excel export, delete with a confirm prompt
Admin overview	`pages/admin/dashboard.tsx`	Stats and a "trigger ingestion" button
User management	`pages/admin/users.tsx`	User table, "add user" dialog (email, password, role), delete
Audit logs	`pages/admin/audit-logs.tsx`	15 rows per page with a record counter

Calculator wizard (`pages/calculator.tsx`):

1. **Compute.** Choose provider, then region, then instance type (labelled with OS, vCPU and memory), plus a quantity. Rows are added to a table and can be removed. Dependent dropdowns are enabled in order.
2. **Storage.** The same pattern with provider, region, storage type and size in GB.
3. **Duration.** One tile of 1, 3, 6, 12, 24 or 36 months (default 12), then the "Calculate Costs" button. It shows a warning toast if nothing was added.
4. **Results.** Three provider cards (monthly total, compute, storage, annual estimate, a cheapest badge), a stacked bar chart of compute and storage cost per provider, buttons for PDF, Excel and Start Over.

The client sends only IDs and quantities. The server looks up the prices.

Exports (`lib/export-utils.ts`): `formatCurrency` formats USD. `exportCalculationToPDF` makes a PDF titled "AWS Calcus - Pricing Intelligence Report" with a generated-on line, calculation ID, duration, a provider summary table and a "Recommended Provider" line, saved as `calcus-report-<id>.pdf`. `exportCalculationToExcel` writes a "Pricing Comparison" sheet saved as `calcus-data-<id>.xlsx`.

Styling: dark theme with a blue primary colour, "glass-card" panels, Inter body font, Syne display font (`index.css`).

6. Pricing engine logic

For one request:

1. Load all providers into an id-to-name map.
2. For each compute selection, fetch its `compute_pricing` row by `compute_pricing_id`; add `price_per_month x quantity` (quantity defaults to 1) to that provider's compute subtotal.

3. For each storage selection, fetch its `storage_pricing` row; add `price_per_gb_month x size_gb` to that provider's storage subtotal.
4. For AWS, Azure and GCP that have at least one subtotal, build `{provider_name, compute_cost_monthly, storage_cost_monthly, total_cost_monthly, total_cost_annual = monthly x 12, is_cheapest}`.
5. If none exist, return `400 No valid pricing data found`.
6. The provider with the lowest monthly total gets `is_cheapest = true`.
7. Insert the row: JSON snapshots, `cheapest_provider`, totals to two decimals, `duration_months`.

Behaviour to know:

- A provider appears in the comparison only if the user picked at least one item from it, because selections are priced as chosen. The UI text promises that equivalents are mapped across all providers, but the code does not do this mapping.
- `duration_months` is stored and shown but does not change any cost figure.
- Pricing rows are fetched one at a time inside loops (N+1 queries).
- Unknown IDs are silently skipped.

7. Configuration and environment

Variable	Used by	Meaning
`DATABASE_URL`	db package, seed, drizzle-kit	PostgreSQL connection string (required)
`PORT`	API server and Vite	Listening port (required)
`BASE_PATH`	Vite	Public base path, `/` as configured (required)
`JWT_SECRET`	API server	Token signing key; defaults to a hard-coded string if unset
`NODE_ENV`	API	`development` for dev script, forced to `production` in the bundle
`REPL_ID`	Vite config	Enables Replit dev plugins when present

Replit artifact configuration: the API listens on port 8080 under `/api` with health check `/api/healthz`; the web app on port 20490 under `/`, built to `dist/public` and served statically with a rewrite of `/*` to `/index.html`.

8. Setup, run and deployment

1. Install Node.js 24 and pnpm. Provide a PostgreSQL database and set `DATABASE_URL`.
2. `pnpm install` (npm and yarn are blocked).
3. Create the schema: `pnpm --filter @workspace/db run push`.
4. Seed once: `pnpm --filter @workspace/scripts run seed`.
5. Start the API: set `PORT=8080` and run `pnpm --filter @workspace/api-server run dev`.
6. Start the web app: set `PORT=20490` and `BASE_PATH=/`, run `pnpm --filter @workspace/aws-calcus run dev`. Both must be reachable under one origin so that `/api` resolves (the Replit router does this; locally, add a proxy).
7. Sign in with the seeded admin account, then change its password.
8. Production: `pnpm run build` (type-checks, then builds every package). Run the API with `node artifacts/api-server/dist/index.cjs` and serve `artifacts/aws-calcus/dist/public` as static files with an SPA fallback.
9. Useful scripts: `pnpm run typecheck`, `pnpm --filter @workspace/db run push-force`. After a merge on Replit, `scripts/post-merge.sh` reinstalls dependencies and pushes the schema.

To regenerate the API client after editing `openapi.yaml`, run Orval with `lib/api-spec/orval.config.ts`.

9. Security observations

- **Default secret.** If `JWT_SECRET` is not set the server signs tokens with a known fallback string. Always set it.
- **Seeded credentials** are weak and documented. Remove or change them in any real deployment.
- **Open CORS.** `cors()` is called with no options.
- **Token storage.** The JWT sits in `localStorage`, readable by any script on the page.
- **No rate limiting or lockout** on login. Failed logins are audited but not throttled.
- **No server-side schema validation.** Request bodies are checked by hand (presence only). Roles are not checked against the enum before insert; the database rejects invalid values with a generic server error. The generated Zod schemas are not used on the server.
- **Password policy.** None.
- **Register endpoint duplicates user creation.** `/v1/auth/register` (admin only) and `/v1/admin/users` do the same job.
- **Audit gaps.** Export actions, the user list view and failed authorisation are not logged. `input_data` is never filled. The IP comes from `x-forwarded-for`, which a client can forge unless a trusted proxy overwrites it.
- **Delete behaviour.** Deleting a user who has calculations or audit rows would violate foreign keys, since no cascade is defined; the request would fail with a server error.
- **Good practices present:** bcrypt cost 12, identical error for wrong e-mail or password, owner-or-admin checks on calculations, self-delete protection, admin routes protected on the server as well as the client.

10. Known limitations and suggested improvements

Area	Observation	Suggestion
Cross-cloud comparison	Only providers the user selected are compared	Build an instance-equivalence map (by vCPU and memory) and price all three providers for every item
Duration	Not used in totals	Multiply by months and show a commitment total
Pricing data	Static seed values, same price in every region; "ingestion" is a stub	Implement scheduled ingestion from the AWS, Azure and GCP price APIs
Seed	Duplicates price rows on repeat	Add unique indexes
Performance	N+1 queries during calculation	Fetch all price rows with one `IN` query
Dashboard	Hard-coded trend labels ("+12% this month", "Synced 2h ago"); "avg" shows AWS only; data points shown as thousands (0.3k)	Compute real trends and show the true count
Validation	Manual checks	Use the generated Zod schemas in the routes
Migrations	`push` only	Adopt versioned migrations
Tests	None in the repository	Add API and calculation unit tests
Stack vs brief	Brief asked for Python/FastAPI and MySQL; built with Node/Express and PostgreSQL	Confirm the deviation with stakeholders
Deployment	Configured for Replit	Add Docker Compose and a reverse proxy for the on-premises target
Export	Exports are not audited; PDF has no per-item detail	Log exports; add selection details to reports